

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего
образования

«Комсомольский-на-Амуре государственный университет»
Кафедра «Проектирование, управление и разработка информационных систем»
Направление 09.03.01 – «Информатика и вычислительная техника»

ПРОГРАММНАЯ ДОКУМЕНТАЦИЯ

Руководство пользователя и программиста

веб-сервиса «Фотоэталон»

к выпускной квалификационной работе

«Разработка веб-приложения для контроля качества изделий по фотоэталонам»

Студент группы 2ИТб-1

В. А. Хитяев

Руководитель

М. Е. Щелкунова

Содержание

1	Руководство пользователя	4
1.1	Назначение раздела	4
1.2	Условия выполнения программы	4
1.3	Запуск приложения пользователем	4
1.4	Работа с группами изделий	4
1.4.1	Создание группы изделия	5
1.4.2	Редактирование группы изделия	6
1.4.3	Удаление группы изделия	6
1.5	Работа с эталонами	6
1.5.1	Создание эталона	6
1.5.2	Редактирование эталона	7
1.6	Загрузка изображений эталона	7
1.7	Выбор основного эталонного изображения	8
1.8	Работа с группами классов и классами объектов	9
1.8.1	Создание группы классов	9
1.8.2	Создание класса объекта	9
1.9	Полигональная разметка изображений	10
1.9.1	Рекомендации по качеству разметки	11
1.10	Обучение модели	11
1.10.1	Управление обученными моделями	12
1.10.2	Импорт и экспорт моделей	13
1.11	Работа с IP-камерами	14
1.12	Контроль изделия	15
1.12.1	Контроль по загруженному изображению	16
1.12.2	Контроль по снимку с камеры	16
1.12.3	Контроль по видеопотоку	17
1.13	Просмотр результата проверки	17
1.14	Сохранение результата проверки	18
1.15	Просмотр истории проверок	18
1.16	Сообщения пользователю	19
2	Руководство программиста	20
2.1	Назначение раздела	20
2.2	Архитектура системы	20
2.3	Структура проекта	21
2.4	Структура клиентской части	21
2.5	Структура серверной части	22
2.6	Требования к окружению разработчика	22

2.7	Установка PostgreSQL на Arch Linux	23
2.8	Создание базы данных	23
2.9	Настройка переменных окружения	23
2.10	Описание основных переменных окружения	24
2.10.1	Общие настройки приложения	24
2.10.2	База данных и файловое хранилище	24
2.10.3	Контроль и live-режим	24
2.10.4	SAM2 для интерактивной сегментации	25
2.10.5	Выравнивание эталона и проверяемого изображения	25
2.10.6	YOLO	25
2.10.7	Встроенный обработчик фоновых задач	25
2.11	Установка зависимостей серверной части	27
2.12	Установка зависимостей клиентской части	27
2.13	Инициализация и миграции базы данных	27
2.14	Локальный запуск проекта	27
2.15	Запуск на сервере предприятия	28
2.15.1	Подготовка backend	29
2.15.2	Пример systemd-сервиса backend	29
2.15.3	Подготовка frontend	30
2.15.4	Пример nginx-конфигурации	30
2.15.5	Проверка production-запуска	32
2.16	Отдельный запуск backend	32
2.17	Отдельный запуск frontend	32
2.18	Проверка работоспособности backend	32
2.19	Проверка работоспособности frontend	33
2.20	Проверка сборки клиентской части	33
2.21	Проверка качества кода клиентской части	33
2.22	Файловое хранилище	33
2.23	Работа с базой данных через GUI	34
2.24	Типовые ошибки и способы устранения	34
2.24.1	PostgreSQL не запущен	34
2.24.2	База данных не существует	34
2.24.3	Неверный пароль PostgreSQL	34
2.24.4	Занят порт backend	35
2.24.5	Занят порт frontend	35
2.24.6	Не запускается обучение модели	35
2.24.7	Не работает камера	35

1 Руководство пользователя

1.1 Назначение раздела

Данный раздел описывает действия оператора при работе с веб-приложением. Оператор - пользователь, который создает группы изделий, добавляет эталоны, загружает изображения, выполняет разметку, запускает обучение модели и проводит проверку изделий.

1.2 Условия выполнения программы

Для работы с веб-приложением необходимы:

- персональный компьютер, ноутбук, планшет или смартфон;
- современный веб-браузер с поддержкой HTML5, JavaScript и мультимедийных потоков;
- доступ к локальной сети или сети Интернет, в которой доступен сервер приложения;
- при проверке по камере - доступная IP-камера.

Установка дополнительного программного обеспечения на рабочем месте пользователя не требуется, так как работа осуществляется через браузерный интерфейс.

1.3 Запуск приложения пользователем

Для запуска приложения пользователю необходимо открыть браузер и перейти по адресу веб-сервиса. Если приложение развернуто на сервере предприятия, вместо локального адреса используется сетевой адрес сервера.

В текущей версии приложения отдельный экран входа не используется: после открытия адреса отображается основной интерфейс с разделами «Группы», «Обучение», «Контроль», «История», «Камеры» и «Настройки».

Место для скриншота

Файл: 01-browser-open.png

Снять браузер с открытым адресом веб-сервиса и загруженной стартовой страницей приложения.

Рис. 1. Открытие веб-сервиса в браузере

1.4 Работа с группами изделий

Группа изделия является основным объектом хранения данных. Внутри группы объединяются эталоны, фотографии, классы объектов, аннотации разметки, обученные модели и результаты проверок, относящиеся к одному изделию или агрегату.

Примеры групп изделий:

- агрегат крепления;
- панель управления;
- блок трубопровода;
- узел соединения;
- изделие определенного типа или модификации.

1.4.1 Создание группы изделия

Для создания группы необходимо выполнить следующие действия:

- 1 Перейти в раздел «Группы».
- 2 Нажать кнопку создания новой группы.
- 3 Заполнить поле «Название».
- 4 При необходимости заполнить поле «Описание».
- 5 Нажать кнопку сохранения.
- 6 Убедиться, что новая группа появилась в списке.

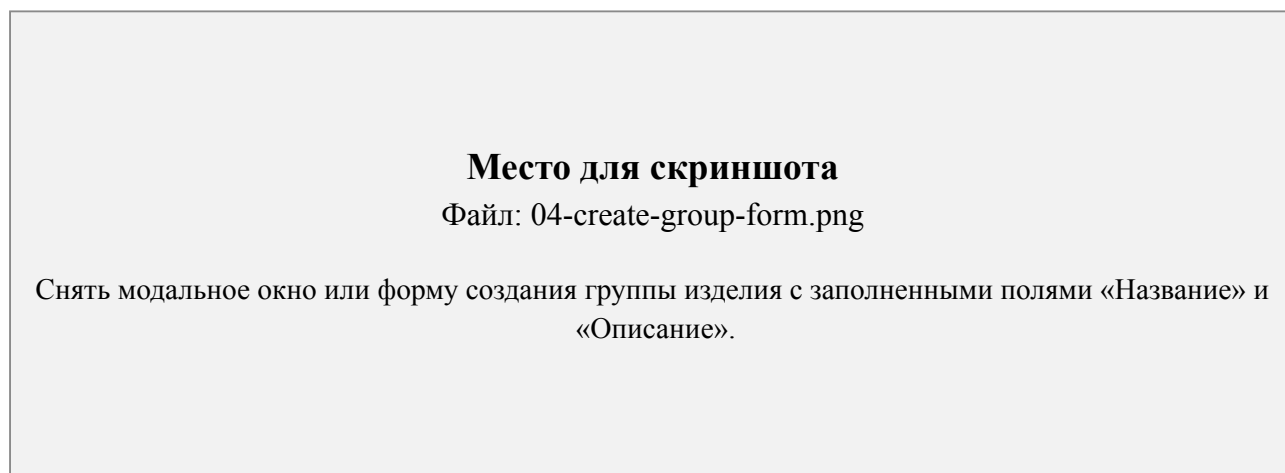


Рис. 2. Создание группы изделия

Место для скриншота

Файл: 05-groups-list.png

Снять список групп после создания новой группы. На скриншоте должна быть видна карточка созданной группы.

Рис. 3. Список групп изделий

1.4.2 Редактирование группы изделия

Для изменения ранее созданной группы пользователь выбирает нужную группу в списке, открывает действие редактирования, изменяет название или описание и сохраняет данные. После сохранения новые значения отображаются в интерфейсе.

1.4.3 Удаление группы изделия

Удаление группы выполняется только в том случае, если пользователь уверен, что связанные с группой эталоны, изображения, разметка, модели и результаты проверок больше не нужны. Перед удалением система может запросить подтверждение действия.

1.5 Работа с эталонами

Эталон представляет собой набор фотографий правильно собранного изделия, сделанных с определенного ракурса. В одной группе может быть несколько эталонов, например: вид спереди, вид сверху, вид сбоку, вид сзади.

1.5.1 Создание эталона

Порядок действий:

- 1 Открыть нужную группу изделия.
- 2 Нажать кнопку создания эталона.
- 3 Указать название эталона.
- 4 Указать ракурс съемки.
- 5 При необходимости заполнить описание.
- 6 Сохранить эталон.

Место для скриншота

Файл: 06-standard-list.png

Снять страницу выбранной группы, где отображается список эталонов изделия.

Рис. 4. Эталон выбранной группы изделия

Место для скриншота

Файл: 07-create-standard-form.png

Снять форму создания эталона с полями названия, описания и ракурса съемки.

Рис. 5. Создание эталона изделия

1.5.2 Редактирование эталона

Для редактирования эталона необходимо открыть карточку эталона, выбрать действие изменения, скорректировать название, описание или ракурс и сохранить изменения.

1.6 Загрузка изображений эталона

После создания эталона пользователь загружает фотографии правильно собранного изделия. Желательно загружать несколько изображений одного ракурса, чтобы в дальнейшем выбрать наиболее удачное эталонное изображение.

Порядок действий:

- 1 Открыть карточку эталона.
- 2 Нажать кнопку добавления изображений.
- 3 Выбрать один или несколько файлов.
- 4 Подтвердить загрузку.
- 5 Дождаться завершения загрузки.
- 6 Проверить наличие миниатюр изображений в карточке эталона.



Рис. 6. Загрузка изображений эталона



Рис. 7. Загруженные изображения эталона

1.7 Выбор основного эталонного изображения

Одно из изображений эталона назначается основным. Основное изображение используется как наиболее удачный вариант ракурса и применяется при дальнейшей разметке и проверке.

Порядок действий:

- 1 Открыть карточку эталона.
- 2 Найти подходящее изображение.
- 3 Нажать кнопку назначения изображения основным.
- 4 Убедиться, что у изображения появилась соответствующая отметка.

Место для скриншота

Файл: 10-reference-image.png

Снять карточку эталона, где у одного изображения отображается отметка эталонного или основного фото.

Рис. 8. Назначение основного изображения эталона

1.8 Работа с группами классов и классами объектов

Перед разметкой изображений необходимо создать группы классов и классы объектов.

Группа классов объединяет однотипные элементы изделия. Класс объекта описывает конкретный контролируемый элемент.

Пример структуры классов:

Группа классов: Крепежные элементы

Класс: Болт М6

Класс: Болт М8

Класс: Гайка М8

Группа классов: Кронштейны

Класс: Кронштейн левый

Класс: Кронштейн правый

1.8.1 Создание группы классов

Порядок действий:

- 1 Открыть раздел классов сегментации выбранной группы.
- 2 Нажать кнопку добавления категории или группы классов.
- 3 Указать название группы.

1.8.2 Создание класса объекта

Порядок действий:

- 1 Выбрать группу классов.
- 2 Нажать кнопку добавления класса.
- 3 Указать название класса объекта.
- 4 При необходимости выбрать цвет отображения.
- 5 Сохранить класс.

Место для скриншота

Файл: 11-segment-classes.png

Снять окно управления группами классов и классами объектов. Должны быть видны группа классов и несколько классов внутри нее.

Рис. 9. Управление классами объектов

1.9 Полигональная разметка изображений

Разметка выполняется в редакторе изображения. Пользователь выбирает класс объекта и отмечает на изображении область, относящуюся к этому классу, с помощью полигона.

Перед началом разметки необходимо убедиться, что:

- изображение загружено;
- создана группа классов;
- создан хотя бы один класс объекта;
- выбран нужный класс объекта;
- открыт редактор разметки.

Порядок выполнения разметки:

- 1 Открыть изображение эталона.
- 2 Выбрать класс объекта в панели классов.
- 3 Включить инструмент полигона.
- 4 Последовательно расставить точки по границе детали.
- 5 Завершить построение полигона.
- 6 Проверить, что полигон появился на изображении и в списке разметки.
- 7 Сохранить результат, если сохранение не выполняется автоматически.

Место для скриншота
Файл: 12-annotation-editor.png

Снять редактор разметки: изображение изделия, список классов, выбранный класс и построенный полигон.

Рис. 10. Полигональная разметка изображения
1.9.1 Рекомендации по качеству разметки

При разметке необходимо придерживаться следующих правил:

- полигон должен проходить по границам объекта;
- не следует включать в полигон лишний фон;
- одинаковые детали должны размечаться одним и тем же классом;
- слишком мелкие и случайные полигоны следует удалять;
- после разметки рекомендуется проверить все изображения эталона.

Качество разметки напрямую влияет на качество обучения модели и корректность последующей проверки изделий.

1.10 Обучение модели

После загрузки и разметки изображений пользователь может запустить обучение модели YOLO. Обучение выполняется по выбранной группе изделия.

Для запуска обучения необходимо наличие:

- группы изделия;
- эталона внутри группы;
- загруженных изображений;
- созданных классов объектов;
- сохраненных полигонов разметки;
- базовых весов модели;
- доступных вычислительных ресурсов сервера.

Порядок действий:

- 1 Перейти в раздел моделей или обучения.

- 2 Выбрать группу изделия.
- 3 Нажать кнопку «Запустить обучение».
- 4 Выбрать архитектуру модели.
- 5 Указать размер изображения.
- 6 Задать количество эпох.
- 7 Задать размер batch size.
- 8 Проверить распределение train, val и test.
- 9 Запустить обучение.
- 10 Дождаться появления задачи обучения в списке моделей.

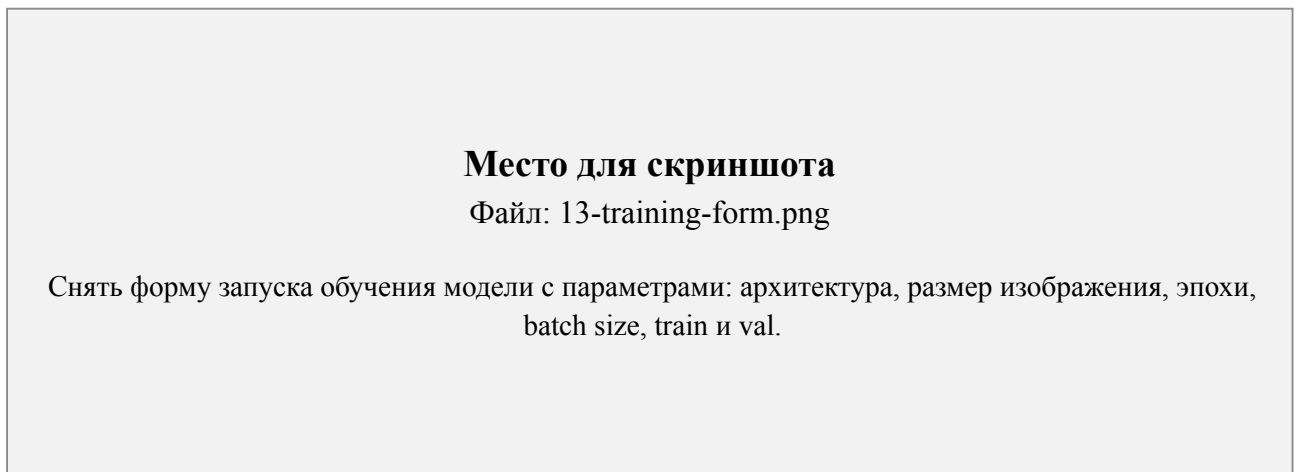


Рис. 11. Настройка параметров обучения модели

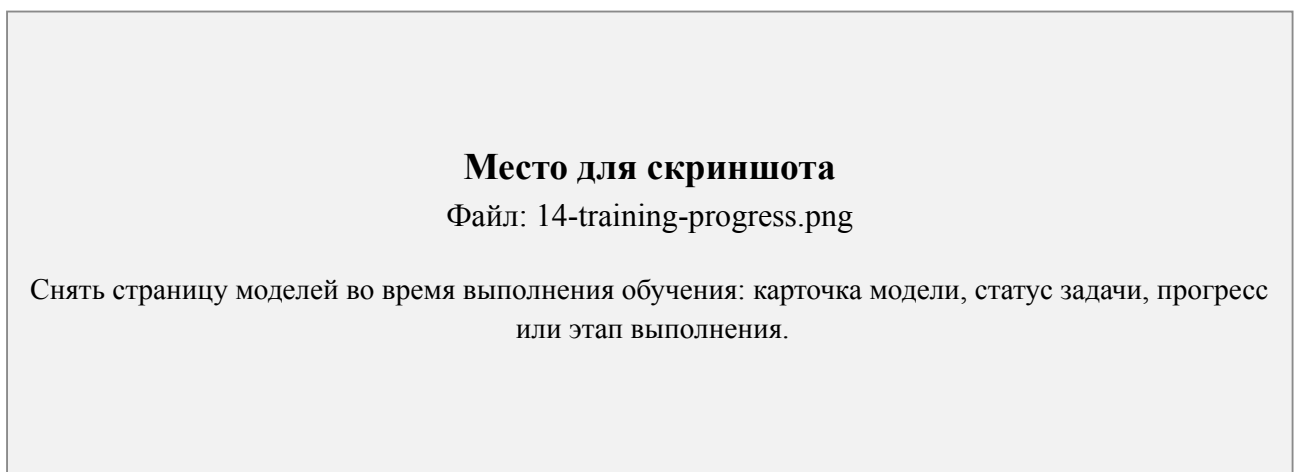


Рис. 12. Отображение процесса обучения модели

1.10.1 Управление обученными моделями

После завершения обучения новая версия модели появляется в списке. Пользователь может просмотреть параметры обучения, метрики, количество классов, дату обучения и статус модели.

Для использования модели при проверке ее необходимо сделать активной, если в интерфейсе предусмотрено ручное управление активной моделью.

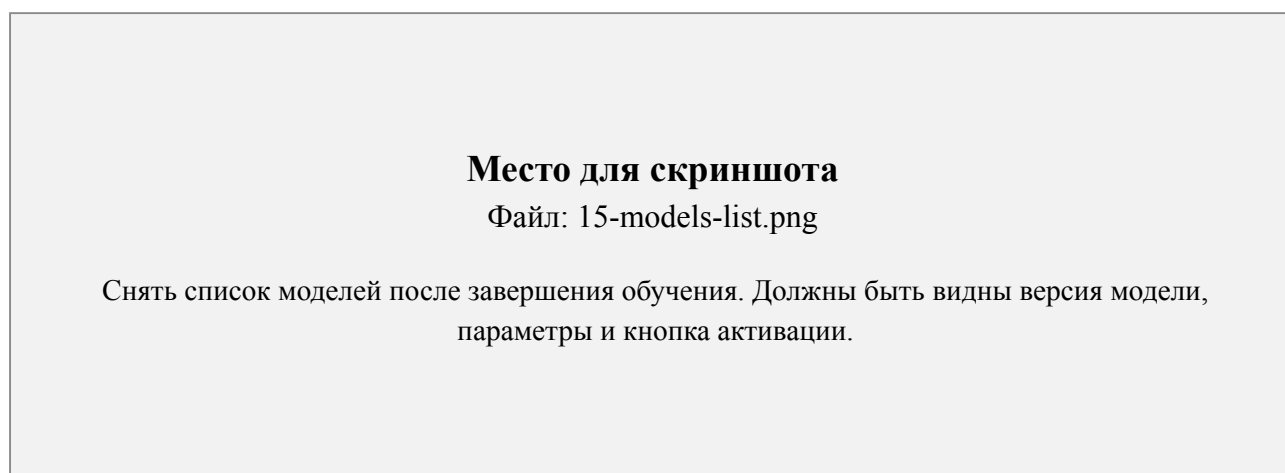


Рис. 13. Список обученных моделей

1.10.2 Импорт и экспорт моделей

Раздел обучения поддерживает перенос моделей между установками системы. Пользователь может импортировать файл весов YOLO в формате .pt, выбрать архитектуру и размер изображения, затем сопоставить классы модели с существующими классами группы или создать новые классы. После успешного импорта модель появляется в списке моделей выбранной группы и может быть активирована для контроля.

Экспорт доступен для готовых версионных моделей, у которых сохранены веса и описание классов. При экспорте система формирует архив с весами модели и файлом классов. Такой архив можно использовать для резервного копирования или переноса модели в другую установку приложения.

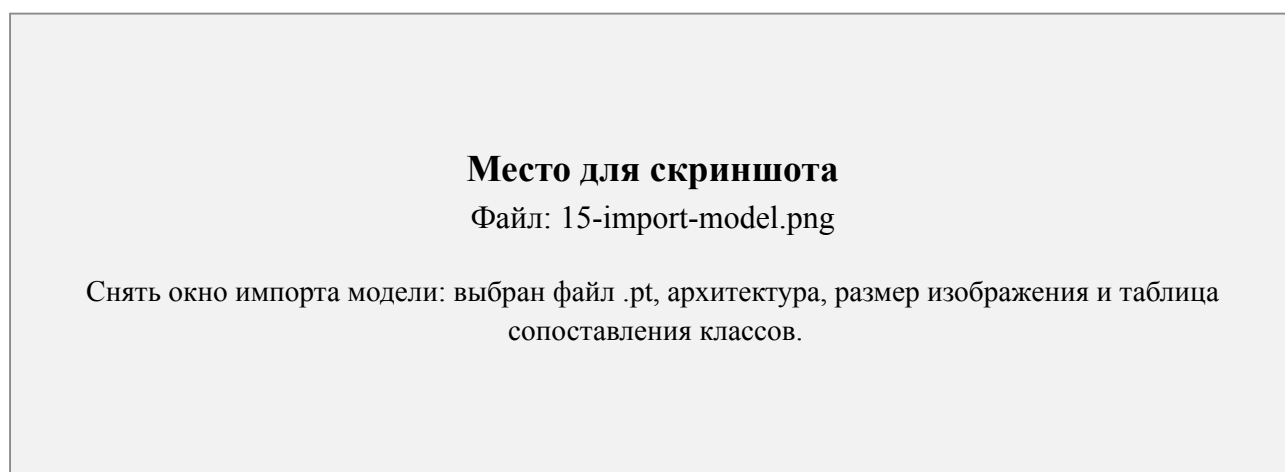


Рис. 14. Импорт обученной модели

Место для скриншота

Файл: 15-export-model.png

Снять окно экспорта модели: выбранная готовая модель, сведения о версии, архитектуре, количестве классов и кнопка скачивания архива.

Рис. 15. Экспорт обученной модели

1.11 Работа с IP-камерами

Для проверки изделий по снимку или видеопотоку необходимо добавить источник изображения. Источником может быть IP-камера, RTSP-поток, HTTP-поток или камера пользовательского устройства, если такой режим предусмотрен системой.

Порядок добавления камеры:

- 1 Перейти в раздел «Камеры».
- 2 Нажать кнопку создания новой камеры.
- 3 Указать название камеры.
- 4 Выбрать протокол подключения.
- 5 Указать адрес потока или параметры подключения.
- 6 Сохранить камеру.
- 7 Проверить доступность камеры.

Место для скриншота

Файл: 16-cameras-list.png

Снять раздел управления камерами со списком добавленных источников изображения.

Рис. 16. Управление IP-камерами

Место для скриншота

Файл: 17-create-camera-form.png

Снять форму добавления камеры с полями названия, протокола и адреса подключения.

Рис. 17. Добавление IP-камеры

1.12 Контроль изделия

Контроль изделия выполняется относительно выбранного эталона и выбранных классов объектов. Система определяет найденные и отсутствующие элементы, после чего формирует результат контроля.

Перед запуском контроля необходимо:

- выбрать группу изделия;
- выбрать эталон;
- выбрать классы объектов для контроля;
- убедиться в наличии активной модели;
- выбрать режим контроля.

В системе предусмотрены три режима проверки:

- проверка по загруженному изображению;
- проверка по снимку с камеры;
- проверка по видеопотоку.

Место для скриншота

Файл: 18-inspection-settings.png

Снять страницу контроля изделия: выбранная группа, эталон, классы объектов и режим контроля.

Рис. 18. Настройка контроля изделия

1.12.1 Контроль по загруженному изображению

Порядок действий:

- 1 Перейти в раздел «Контроль».
- 2 Выбрать режим «Загрузка фото».
- 3 Выбрать группу и эталон.
- 4 Выбрать классы объектов.
- 5 Загрузить изображение проверяемого изделия.
- 6 Нажать кнопку запуска контроля.
- 7 Дождаться завершения обработки.
- 8 Просмотреть результат.



1.12.2 Контроль по снимку с камеры

Порядок действий:

- 1 Перейти в раздел «Контроль».
- 2 Выбрать режим «Снимок с камеры».
- 3 Выбрать камеру.
- 4 Получить кадр с камеры.
- 5 Запустить контроль.
- 6 Просмотреть результат.

Место для скриншота

Файл: 20-inspection-snapshot.png

Снять режим контроля по снимку с камеры: выбран источник изображения и отображается кадр или предпросмотр.

Рис. 20. Контроль изделия по снимку с камеры **1.12.3 Контроль по видеопотоку**

Порядок действий:

- 1 Перейти в раздел «Контроль».
- 2 Выбрать режим «Live-режим».
- 3 Выбрать камеру.
- 4 Запустить поток.
- 5 Наблюдать результат проверки в реальном времени.
- 6 При необходимости сохранить текущий кадр или остановить проверку.

Место для скриншота

Файл: 21-inspection-realtime.png

Снять режим контроля по видеопотоку: видео с камеры и панель текущего результата.

Рис. 21. Контроль изделия по видеопотоку **1.13 Просмотр результата проверки**

После завершения проверки система отображает:

- общий статус проверки;
- количество найденных объектов;
- количество ожидаемых объектов;
- список найденных элементов;

- список отсутствующих элементов;
- список лишних элементов, если они обнаружены;
- изображение или кадр с визуальным отображением результата;
- сведения об использованной модели.

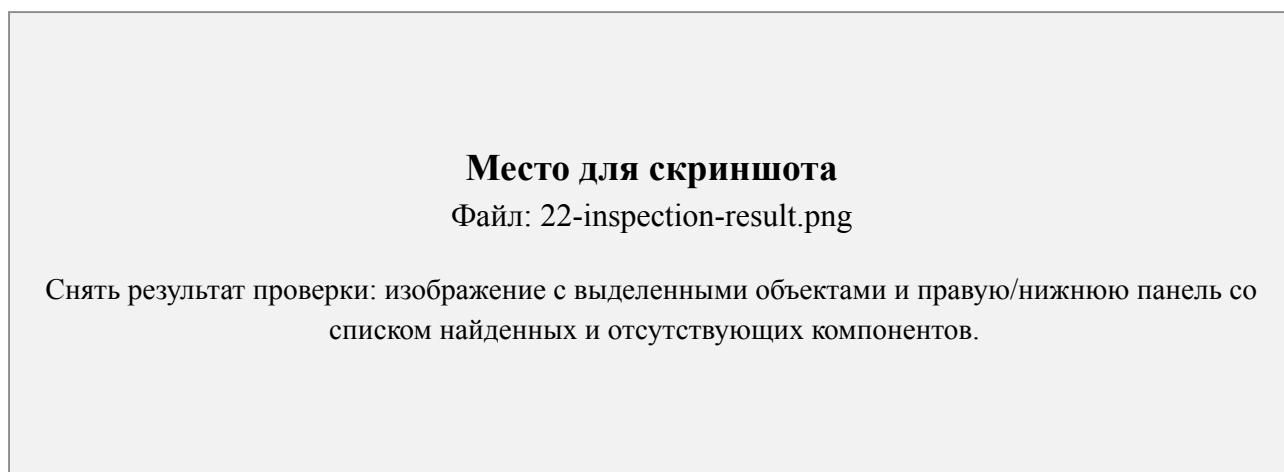


Рис. 22. Результат проверки изделия

1.14 Сохранение результата проверки

Если результат проверки необходимо сохранить, пользователь заполняет комментарий и нажимает кнопку сохранения. После сохранения результат становится доступен в истории проверок.



Рис. 23. Сохранение результата проверки

1.15 Просмотр истории проверок

История проверок предназначена для просмотра ранее сохраненных результатов контроля. В списке отображаются дата проверки, группа, эталон, режим контроля, итоговый статус и дополнительные сведения.

Порядок действий:

- 1 Перейти в раздел «История».
- 2 Найти нужную проверку в списке.

- 3 При необходимости использовать фильтры.
- 4 Открыть карточку проверки.
- 5 Просмотреть изображение результата и список компонентов.



Рис. 24. История выполненных проверок

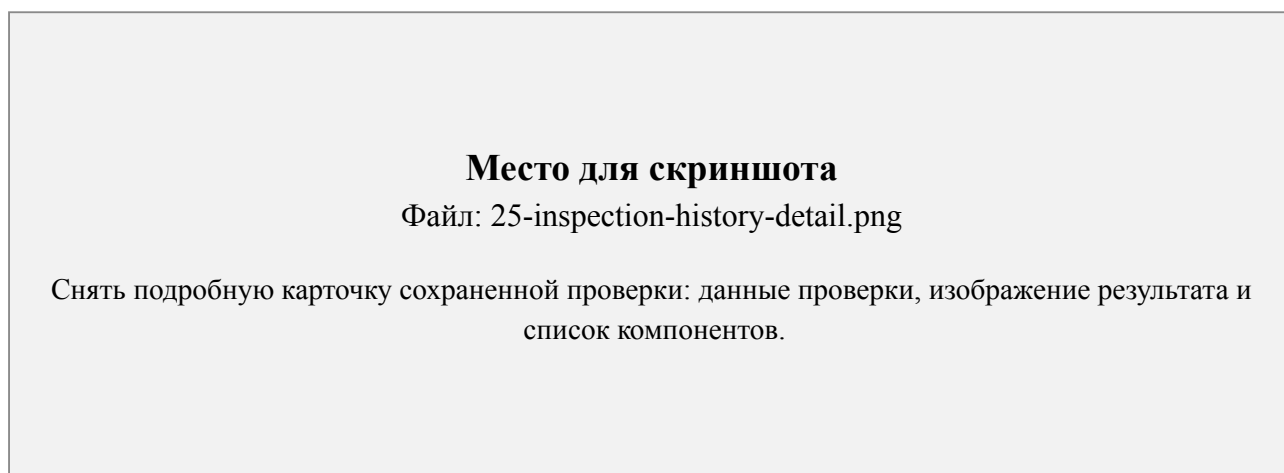


Рис. 25. Просмотр сохраненного результата проверки

1.16 Сообщения пользователю

В процессе работы система отображает информационные сообщения, предупреждения и ошибки. Основные типы сообщений приведены в таблице.

Событие	Сообщение	Действие пользователя
Не заполнено обязательное поле	Поле обязательно	Заполнить поле и повторить действие
Не найден объект	Не найдено	Обновить страницу или выбрать другой объект

Недостаточно данных для обучения	Для обучения нужны эталоны, фото, классы и разметка	Проверить подготовку данных
Ошибка подключения камеры	Не удалось получить кадр	Проверить адрес камеры и сеть
Ошибка проверки	Проверка прервана из-за ошибки	Повторить действие или обратиться к администратору

2 Руководство программиста

2.1 Назначение раздела

Данный раздел предназначен для программиста или администратора, который выполняет установку, настройку, запуск и сопровождение веб-сервиса «Фотоэталон».

2.2 Архитектура системы

Веб-сервис имеет клиент-серверную архитектуру. Пользователь взаимодействует с системой через браузер. Клиентская часть отправляет HTTP-запросы к серверной части. Серверная часть выполняет бизнес-логику, взаимодействует с базой данных, файловым хранилищем, модулем фоновых задач и модулями машинного зрения.

Основные компоненты системы:

- клиентское приложение React;
- серверное приложение FastAPI;
- PostgreSQL;
- файловое хранилище;
- очередь и фоновые задачи;
- модуль обучения YOLO;
- модуль проверки изделий;
- модуль камер.

Место для скриншота

Файл: 26-system-architecture.png

Подготовить схему архитектуры: браузер, React, FastAPI, PostgreSQL, файловое хранилище, очередь задач, YOLO, камеры.

Рис. 26. Архитектура веб-сервиса

2.3 Структура проекта

Типовая структура проекта:

```
photo-standards-db/  
  client/           # клиентская часть React  
  server/           # серверная часть FastAPI  
  storage/          # файловое хранилище  
  docs/             # документация  
  deploy/           # скрипты и шаблоны для запуска и развертывания  
  package.json      # команды проекта
```

Каталог `deploy/` содержит готовые части для запуска проекта: шаблон переменных окружения, `nginx`-конфигурации, пример `systemd`-сервиса, `dev`-лаунчер `deploy/scripts/run-local.js` и кроссплатформенный `Node`-скрипт для инициализации базы данных, `dev`-запуска и `production-like` проверки. Основные команды вынесены в `package.json`, поэтому их можно запускать одинаково на `Linux` и

`Windows` через `npm run ....`

2.4 Структура клиентской части

Клиентская часть расположена в каталоге `client/`.

Основные каталоги:

```
client/src/  
  app/              # маршрутизация и общая сборка приложения  
  components/       # общие UI-компоненты  
  page-components/  # функциональные компоненты страниц  
  hooks/            # общие hooks  
  lib/              # api-client, query-client, query-keys  
  types/            # типы контрактов  
  utils/            # общие функции  
  styles/           # общие стили
```

Место для скриншота

Файл: 27-client-structure.png

Снять дерево каталогов клиентской части в IDE или файловом менеджере.

2.5 Структура серверной части

Рис. 27. Структура клиентской части проекта

Серверная часть расположена в каталоге `server/`.

Основные каталоги:

```
server/src/  
  app/           # настройка FastAPI, база данных, зависимости  
  infra/         # инфраструктура: очередь, storage, runtime  
  modules/       # бизнес-модули приложения  
  constants.py   # общие константы  
  main.py        # точка входа backend
```

Место для скриншота

Файл: 28-server-structure.png

Снять дерево каталогов серверной части в IDE или файловом менеджере.

2.6 Требования к окружению разработчика

Рис. 28. Структура серверной части проекта

Для локального запуска и сопровождения проекта рекомендуется следующее программное обеспечение:

- Linux или Windows 10 и выше;
- Python 3.12 или выше;
- Node.js 20 или выше;
- npm;
- PostgreSQL;

- Git;
- Visual Studio Code, PyCharm или аналогичная IDE;
- DBeaver, pgAdmin или psql для работы с PostgreSQL;
- современный браузер.

Для обучения моделей с ускорением рекомендуется видеокарта NVIDIA и настроенная CUDA-среда. При отсутствии GPU обучение и проверка могут выполняться на CPU, но скорость обработки будет ниже.

Команды `npm run deploy:*` работают через Node.js и могут использоваться как на Linux, так и на Windows при наличии установленных Python, Node.js, PostgreSQL и nginx. При этом шаблон systemd-сервиса и пример установки nginx в `/etc/nginx` относятся к Linux-серверу; на Windows backend запускается той же `npm`-командой или через выбранный администратором механизм служб.

2.7 Установка PostgreSQL на Arch Linux

На Arch Linux PostgreSQL можно установить командой:

```
sudo pacman -S postgresql
```

После установки необходимо инициализировать кластер базы данных, если это еще не выполнено:

```
sudo -iu postgres initdb -D /var/lib/postgres/data
```

Запуск и включение сервиса:

```
sudo systemctl enable --now postgresql
```

Проверка состояния:

```
systemctl is-active postgresql
pg_isready -h localhost -p 5432
```

2.8 Создание базы данных

Для проекта рекомендуется создать отдельную базу данных:

```
sudo -iu postgres createdb "photo-standards"
```

При необходимости можно задать пароль пользователю postgres:

```
sudo -iu postgres psql
ALTER USER postgres WITH PASSWORD 'postgres';
\q
```

Проверка подключения:

```
psql -h localhost -U postgres -d photo-standards -c "SELECT 1;"
```

2.9 Настройка переменных окружения

Файл переменных окружения серверной части располагается по пути:

```
server/.env
```

Минимальный пример подключения через TCP:

```
DB_HOST=localhost
DB_PORT=5432
DB_USER=postgres
DB_PASS=postgres
DB_NAME=photo-standards
```

Пример подключения через Unix socket:

```
DB_SOCKET_DIR=/run/postgresql
DB_USER=postgres
DB_PASS=postgres
DB_NAME=photo-standards
```

2.10 Описание основных переменных окружения

2.10.1 Общие настройки приложения

Переменная	Пример	Назначение
APP_NAME	PhotoStandards API	Название FastAPI-приложения
DEBUG	true	Режим отладки backend
SERVER_HOST	0.0.0.0	Адрес запуска backend
SERVER_PORT	3001	Порт backend
SERVER_RELOAD	true	Автоперезапуск backend при разработке

2.10.2 База данных и файловое хранилище

Переменная	Пример	Назначение
DB_HOST	localhost	Адрес сервера PostgreSQL при TCP-подключении
DB_PORT	5432	Порт PostgreSQL
DB_SOCKET_DIR	/run/postgresql	Путь к Unix socket PostgreSQL
DB_USER	postgres	Имя пользователя базы данных
DB_PASS	postgres	Пароль пользователя базы данных
DB_NAME	photo-standards	Имя базы данных проекта
STORAGE_ROOT	../storage	Корневой каталог файлового хранилища

2.10.3 Контроль и live-режим

Переменная	Пример	Назначение
INSPECTION_VERIFICATION_MODE	alignment	Режим проверки изделия: alignment ИЛИ yolo_count
INSPECTION_POLYGON_PROJECTION_MODE	global	Режим проекции полигонов: global ИЛИ adaptive
MAX_REALTIME_INSPECTIONS	2	Максимальное количество одновременных live-проверок

PHOTOAPP_REALTIME_PROFILE	false	Включение профилирования live-контроля
PHOTOAPP_REALTIME_PROFILE_WINDOW	1	Размер окна статистики профилировщика
PHOTOAPP_REALTIME_PROFILE_SUMMARY_EVERY	3	Период вывода сводки профилировщика

2.10.4 SAM2 для интерактивной сегментации

Переменная	Пример	Назначение
SAM2_DEVICE	cpu	Устройство запуска SAM2
SAM2_ROOT	storage/weights	Каталог весов SAM2
SAM2_MODEL_CFG	configs/sam2.1/ sam2.1_hiera_s.yaml	Конфигурация модели SAM2
SAM2_CHECKPOINT	storage/weights/sam2.1_hiera_s_checkpoint	Путь к checkpoint SAM2

2.10.5 Выравнивание эталона и проверяемого изображения

Переменная	Пример	Назначение
ALIGNMENT_BACKEND	auto	Backend выравнивания: auto, torch ИЛИ orb
ALIGNMENT_DEVICE	auto	Устройство выравнивания: auto, cuda ИЛИ cpu
ALIGNMENT_ORB_FALLBACK	true	Разрешить ORB fallback при недоступности основного backend
ALIGNMENT_IDENTITY_SHORTCUT	true	Разрешить быстрый путь без выравнивания для совпадающих изображений

2.10.6 YOLO

Переменная	Пример	Назначение
YOLO_DEVICE	auto	Устройство запуска YOLO: auto, cuda ИЛИ cpu
YOLO_DEFAULT_IMGSZ	640	Размер изображения YOLO по умолчанию
YOLO_HALF	false	Использование half precision для YOLO

2.10.7 Встроенный обработчик фоновых задач

Переменная	Пример	Назначение
TASK_RUNNER_ENABLED	true	Включение встроенного обработчика фоновых задач

TASK_RUNNER_FAIL_ACTIVE_ON_STARTUP		Помечать активные задачи ошибочными при старте
TASK_RUNNER_POLL_INTERVAL_SEC		Интервал опроса очереди задач
TASK_RUNNER_CPU_CONCURRENCY		Количество CPU-задач одновременно
TASK_RUNNER_GPU_CONCURRENCY		Количество GPU-задач одновременно
TASK_RUNNER_TRAINING_CONCURRENCY		Количество одновременных задач обучения
TASK_RUNNER_INSPECTION_CONCURRENCY		Количество одновременных задач контроля
TASK_RUNNER_MODEL_IMPORT_CONCURRENCY		Количество одновременных задач импорта моделей
TASK_RUNNER_HEARTBEAT_TIMEOUT_SEC		Таймаут heartbeat задачи
TASK_RUNNER_SERVICE_MONITOR_INTERVAL_SEC		Интервал мониторинга служебных процессов
TASK_RUNNER_SERVICE_RESTART_DELAY_SEC		Начальная задержка перезапуска служебного процесса
TASK_RUNNER_SERVICE_MAX_RESTART_DELAY_SEC		Максимальная задержка перезапуска служебного процесса
TASK_RUNNER_SERVICE_STOP_TIMEOUT_SEC		Таймаут остановки служебного процесса
TASK_RUNNER_SERVICE_PARENT_CHECK_INTERVAL_SEC		Интервал проверки родительского процесса
TASK_RUNNER_SERVICE_HEARTBEAT_INTERVAL_SEC		Интервал heartbeat служебного процесса
TASK_RUNNER_SERVICE_HEARTBEAT_TIMEOUT_SEC		Таймаут heartbeat служебного процесса
TASK_RUNNER_SERVICE_MEMORY_LIMIT_MB		Лимит памяти служебного процесса; 0 означает без лимита
TASK_RUNNER_SERVICE_MIN_FREE_MEMORY_MB		Минимальный свободный объем памяти
TASK_RUNNER_SERVICE_CPU_LIMIT_PERCENT		Порог загрузки CPU для контроля ресурсов

TASK_RUNNER_SERVICE_RESOURCE_GRACE_TICKS	Количество допустимых превышений лимитов ресурсов
--	---

2.11 Установка зависимостей серверной части

Перейти в каталог сервера:

```
cd server
```

Создать виртуальное окружение:

```
python -m venv .venv
```

Активировать виртуальное окружение:

```
source .venv/bin/activate
```

Установить зависимости:

```
pip install -r requirements.txt
```

2.12 Установка зависимостей клиентской части

Перейти в каталог клиента:

```
cd client
```

Установить зависимости:

```
npm install
```

2.13 Инициализация и миграции базы данных

Перед запуском проекта необходимо применить миграции базы данных. Из корня проекта выполняется команда:

```
npm run db:migrate
```

То же действие через deploy-скрипт:

```
npm run deploy:init-db
```

Для первичной инициализации может использоваться команда:

```
npm run db:bootstrap
```

Или:

```
npm run deploy:bootstrap-db
```

Очередь фоновых задач работает через таблицы основной базы данных и встроенный `TaskQueueRunner`. Отдельная служебная схема для очереди в текущей конфигурации не требуется.

2.14 Локальный запуск проекта

В проекте предусмотрен локальный запуск клиентской и серверной частей. В зависимости от текущей конфигурации запуск может выполняться из корня проекта или из каталога клиента.

Основной вариант запуска:

```
npm run dev
```

Эквивалентная команда через deploy-скрипт:

```
npm run deploy:dev
```

Если запуск выполняется из каталога `client/`, может использоваться команда:

```
npm run project
```

После запуска обычно доступны адреса:

Frontend: `http://127.0.0.1:5173`

Backend: `http://127.0.0.1:3001`

Локальный лаунчер `deploy/scripts/run-local.js` также поддерживает переменные `CLIENT_HOST`, `CLIENT_PORT`, `SERVER_BASE_URL`, `SERVER_HEALTH_URL`, `SERVER_READY_TIMEOUT_MS`, `LAUNCHER_SHUTDOWN_TIMEOUT_MS` и `RUN_INIT_DB`. Если `RUN_INIT_DB=true`, перед запуском backend выполняется первичная инициализация базы данных.

Место для скриншота

Файл: 29-terminal-start.png

Снять терминал после успешного запуска проекта. Должны быть видны адреса frontend и backend или сообщения о готовности серверов.

Рис. 29. Запуск веб-сервиса в локальной среде

2.15 Запуск на сервере предприятия

Для эксплуатации на сервере предприятия не следует использовать режим разработки с автоперезапуском backend и dev-сервером Vite. Рекомендуемый вариант развертывания:

- PostgreSQL работает как системный сервис;
- backend запускается как постоянный сервис без `SERVER_RELOAD`;
- frontend предварительно собирается командой `npm run build`;
- nginx отдает статические файлы frontend и проксирует `/api/`, `/storage/` и `/ws/` на backend;
- переменные окружения хранятся в `server/.env` или в отдельном `systemd environment file`.

В проекте подготовлен каталог `deploy/`, чтобы не собирать все части вручную:

- `deploy/env/server.env.example` - шаблон переменных окружения backend;

- `deploy/nginx/enterprise.conf` - nginx server block для сервера предприятия;
- `deploy/systemd/photo-standards-backend.service` - пример systemd-сервиса backend;
- `deploy/scripts/run-local.js` - локальный dev-запуск backend и frontend;
- `deploy/scripts/deploy.mjs` - кроссплатформенный runner для deploy-команд;
- `npm run deploy:init-db` - миграции базы данных;
- `npm run deploy:prod:local` - локальная production-like проверка через nginx;
- `npm run deploy:prod:check` - проверка локального production-like запуска;
- `npm run deploy:prod:stop` - остановка локального production-like запуска.

2.15.1 Подготовка backend

На сервере необходимо установить зависимости, создать `server/.env`, применить миграции и проверить запуск backend:

```
cd /opt/photo-standards-db
cd server
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cd ..
npm run db:migrate
```

Для production-запуска backend должны использоваться значения:

```
DEBUG=false
SERVER_HOST=127.0.0.1
SERVER_PORT=3001
SERVER_RELOAD=false
```

Если backend должен принимать подключения не только через локальный nginx, допускается `SERVER_HOST=0.0.0.0`, но в этом случае доступ к порту backend необходимо ограничить сетевыми правилами.

2.15.2 Пример systemd-сервиса backend

Пример файла `/etc/systemd/system/photo-standards-backend.service`:

```
[Unit]
Description=PhotoStandards backend
After=network.target postgresql.service

[Service]
Type=simple
WorkingDirectory=/opt/photo-standards-db/server
Environment=DEBUG=false
Environment=SERVER_HOST=127.0.0.1
Environment=SERVER_PORT=3001
Environment=SERVER_RELOAD=false
ExecStart=/opt/photo-standards-db/server/.venv/bin/python src/main.py
Restart=always
RestartSec=5
```

```
[Install]
WantedBy=multi-user.target
```

Готовый шаблон находится в `deploy/systemd/photo-standards-backend.service`. При установке на предприятии его можно скопировать в `/etc/systemd/system/` и изменить пути, если проект расположен не в `/opt/photo-standards-db`.

После создания файла сервис включается и запускается командами:

```
sudo systemctl daemon-reload
sudo systemctl enable --now photo-standards-backend
sudo systemctl status photo-standards-backend
```

Место для скриншота

Файл: 32-production-backend-service.png

Снять вывод `systemctl status photo-standards-backend`: сервис должен быть `active/running`, без ошибок запуска.

Рис. 30. Сервис backend на сервере предприятия

2.15.3 Подготовка frontend

Frontend в production-режиме собирается заранее:

```
cd /opt/photo-standards-db/client
npm install
npm run build
```

Содержимое каталога `client/dist/` копируется в каталог, из которого `nginx` отдает статические файлы, например:

```
sudo mkdir -p /var/www/photo-standards
sudo rsync -a --delete dist/ /var/www/photo-standards/
```

2.15.4 Пример nginx-конфигурации

Пример `server block` для установки без Docker:

```
server {
    listen 80;
    server_name photo-standards.local;

    root /var/www/photo-standards;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }

    location /api/ {
```

```

        proxy_pass http://127.0.0.1:3001;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location /storage/ {
        proxy_pass http://127.0.0.1:3001;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location /ws/ {
        proxy_pass http://127.0.0.1:3001;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}

```

Готовый вариант этого `server block` находится в `deploy/nginx/enterprise.conf`. Перед использованием необходимо заменить `server_name`, а при необходимости и путь `root`.

Маршрут `/ws/` обязателен для live-обновлений задач, статусов камер и контроля по видеопотоку. После изменения конфигурации необходимо проверить `nginx` и перезагрузить сервис:

```

sudo nginx -t
sudo systemctl reload nginx

```

Для локальной проверки `production`-схемы без изменения системного `/etc/nginx` используется команда:

```

npm run deploy:prod:local

```

Она применяет миграции, собирает frontend, генерирует `nginx`-конфиг из `deploy/nginx/local-test.conf.template`, запускает backend на `127.0.0.1:3011` и `nginx` на `127.0.0.1:8088`. После запуска можно проверить стек командой:

Команда `npm run deploy:prod:local` работает как обычный серверный процесс и занимает текущий терминал. Проверку и остановку удобно выполнять из второго терминала.

```

npm run deploy:prod:check

```

Остановка выполняется командой:

```
npm run deploy:prod:stop
```

Эти команды нужны только для проверки production-схемы на машине разработчика. На сервере предприятия следует использовать конфигурацию из предыдущего примера с реальными путями и доменным именем.

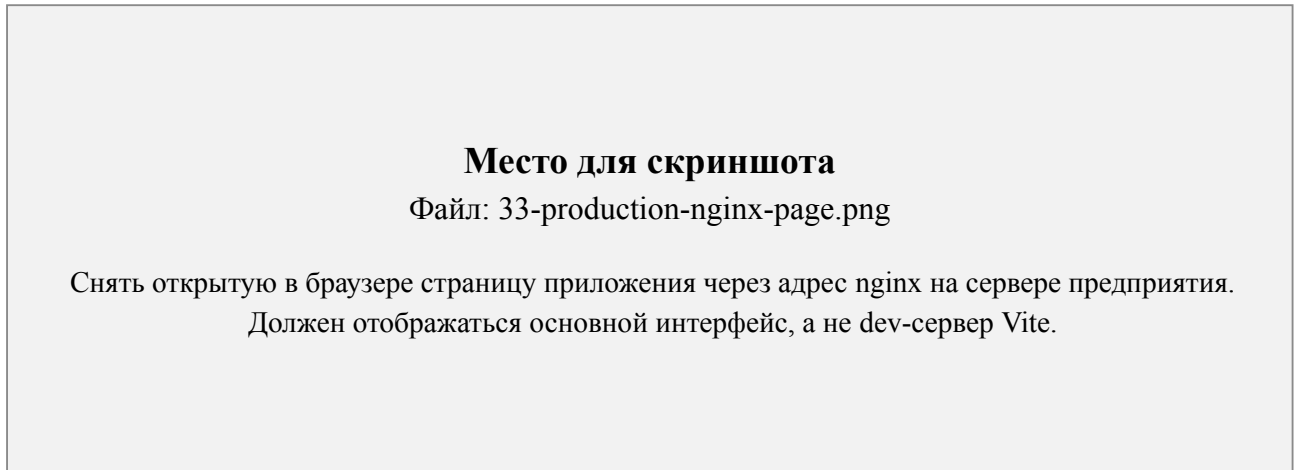


Рис. 31. Frontend через nginx

2.15.5 Проверка production-запуска

После запуска сервисов необходимо проверить:

- 1 backend отвечает на `http://127.0.0.1:3001/api/health`;
- 2 nginx отдает frontend по доменному имени или IP-адресу сервера;
- 3 запросы `/api/` из браузера не возвращают ошибки прокси;
- 4 изображения из `/storage/` открываются через nginx;
- 5 live-обновления и видеопоток работают через `/ws/`;
- 6 после перезагрузки сервера backend автоматически стартует через systemd.

2.16 Отдельный запуск backend

Для отладки серверной части backend можно запускать отдельно:

```
cd server
DEBUG=true SERVER_RELOAD=true SERVER_HOST=0.0.0.0 SERVER_PORT=3001 .venv/bin/
python src/main.py
```

2.17 Отдельный запуск frontend

Для отладки клиентской части frontend можно запускать отдельно:

```
cd client
npm run dev -- --host 0.0.0.0 --port 5173
```

2.18 Проверка работоспособности backend

После запуска серверной части необходимо проверить служебный endpoint состояния:

```
http://127.0.0.1:3001/api/health
```


Если сервер работает корректно, возвращается успешный ответ состояния приложения.

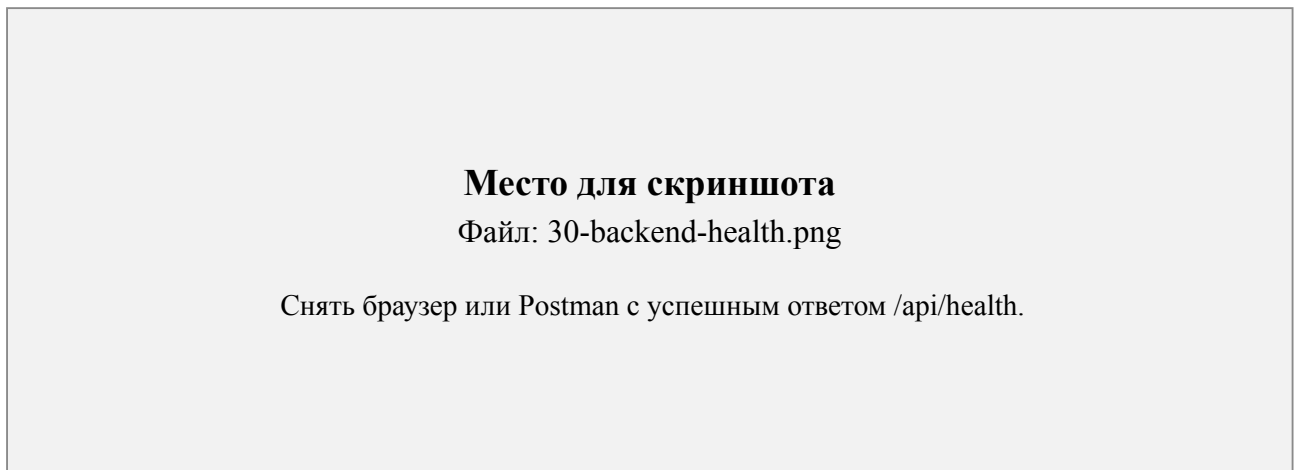


Рис. 32. Проверка работоспособности серверной части

2.19 Проверка работоспособности frontend

Для проверки клиентской части необходимо открыть адрес:

`http://127.0.0.1:5173`

Если клиентская часть работает корректно, в браузере отображается интерфейс веб-сервиса.

2.20 Проверка сборки клиентской части

Для проверки сборки frontend используется команда:

```
cd client
npm run build
```

2.21 Проверка качества кода клиентской части

Для запуска проверки lint используется команда:

```
cd client
npm run lint
```

2.22 Файловое хранилище

Файловое хранилище используется для хранения:

- загруженных изображений эталонов;
- кадров и изображений проверок;
- визуализированных результатов;
- весов моделей;
- временных файлов обучения;
- экспортированных наборов данных или моделей.

Обычно хранилище находится в каталоге:

`storage/`

Удалять каталог `storage/` без резервного копирования не рекомендуется.

2.23 Работа с базой данных через GUI

Для просмотра базы данных удобно использовать DBeaver. Подключение к локальной базе PostgreSQL обычно выполняется со следующими параметрами:

```
Host: localhost
Port: 5432
Database: photo-standards
Username: postgres
Password: postgres
```

Если требуется создавать базы данных через DBeaver, в настройках подключения следует включить параметр `Show all databases`.

Место для скриншота

Файл: 31-dbeaver-connection.png

Снять окно настройки подключения DBeaver к PostgreSQL с host localhost, port 5432 и базой photo-standards.

Рис. 33. Подключение к PostgreSQL через DBeaver

2.24 Типовые ошибки и способы устранения

2.24.1 PostgreSQL не запущен

Проверка:

```
systemctl is-active postgresql
pg_isready -h localhost -p 5432
```

Решение:

```
sudo systemctl enable --now postgresql
```

2.24.2 База данных не существует

Признак ошибки:

```
database "photo-standards" does not exist
```

Решение:

```
sudo -iu postgres createdb "photo-standards"
npm run db:migrate
```

2.24.3 Неверный пароль PostgreSQL

Признак ошибки:

```
password authentication failed for user "postgres"
```

Решение:

```
sudo -iu postgres psql
```

```
ALTER USER postgres WITH PASSWORD 'postgres';  
\q
```

Затем необходимо проверить `server/.env`.

2.24.4 Занят порт backend

Проверка:

```
ss -ltnp | grep 3001
```

Решение: остановить процесс, занимающий порт, или изменить переменную `SERVER_PORT`.

2.24.5 Занят порт frontend

Проверка:

```
ss -ltnp | grep 5173
```

Решение:

```
cd client  
npm run dev -- --host 0.0.0.0 --port 5174
```

2.24.6 Не запускается обучение модели

Возможные причины:

- отсутствуют эталоны;
- отсутствуют изображения;
- отсутствуют классы объектов;
- изображения не размечены;
- отсутствуют базовые веса модели;
- не хватает оперативной памяти или памяти GPU;
- занята очередь фоновых задач.

Рекомендуемые действия:

1 Проверить наличие размеченных изображений.

2 Проверить наличие классов объектов.

3 Уменьшить размер изображения.

4 Уменьшить batch size.

5 Проверить логи backend и встроенного обработчика задач.

2.24.7 Не работает камера

Возможные причины:

- указан неверный URL;
- камера недоступна из сети;

- требуется логин и пароль;
- поток не поддерживается;
- камера занята другим приложением.

Рекомендуемые действия:

- 1 Проверить адрес камеры.
- 2 Проверить доступность камеры из сети сервера.
- 3 Проверить поток через VLC или ffmpeg.
- 4 Изменить параметры подключения.
- 5 Перезапустить backend.